

Nom : BEMBE MOCTAR

ID : 12530131

Nom : Ahmed Charghi

ID : 12530066

Rapport Technique

Algorithme d'Ordonnancement SRTF

Python + MySQL + Docker sur Debian

Module : Systèmes d'Exploitation — SE-LIU-2026

Introduction Générale

1) Contexte :

Dans le domaine des systèmes d'exploitation, la gestion des processus est l'une des fonctions les plus fondamentales. Le système d'exploitation doit décider à tout moment quel processus doit utiliser le processeur. Cette décision est prise par un composant appelé l'ordonnanceur (scheduler).

L'ordonnancement des processus a un impact direct sur les performances du système : temps de réponse, utilisation du processeur, temps d'attente des utilisateurs. Il existe plusieurs algorithmes d'ordonnancement, chacun avec ses avantages et ses inconvénients.

2) Objectif du projet :

Ce projet a pour objectif d'implémenter et de simuler l'algorithme d'ordonnancement SRTF (Shortest Remaining Time First) en utilisant les technologies suivantes :

Python : pour la logique algorithmique

MySQL : pour la persistance et le stockage des résultats

Docker : pour la conteneurisation sur un système Debian

GitHub : pour la gestion de version et le partage du code

1.3 Structure du rapport

Ce rapport est organisé comme suit :

Présentation théorique de l'algorithme SRTF

Description de l'architecture technique du projet

Implémentation et détails du code

Résultats des tests et analyse

Conclusion et perspectives

Théorie de l'Ordonnancement

1) Qu'est-ce que l'ordonnancement ?

L'ordonnancement (scheduling) est le mécanisme par lequel le système d'exploitation alloue le processeur aux différents processus en attente. L'objectif principal est d'optimiser l'utilisation des ressources tout en minimisant les temps d'attente.

2)Critères d'évaluation d'un algorithme:

Un algorithme d'ordonnancement est évalué selon plusieurs critères :

Critère

Définition

Utilisation CPU

Pourcentage du temps où le CPU est occupé

Débit (Throughput)

Nombre de processus terminés par unité de temps

Temps de rotation (Turnaround)

Temps total entre la soumission et la fin d'un processus

Temps d'attente (Waiting Time)

Temps passé dans la file d'attente

Temps de réponse

Temps entre la soumission et la première réponse

2) Types d'ordonnancement:

Il existe deux grandes catégories :

Ordonnancement non-préemptif : Une fois qu'un processus a le CPU, il le garde jusqu'à ce qu'il se termine ou se bloque. Exemple : FCFS, SJF.

Ordonnancement préemptif : Le système peut interrompre un processus en cours pour donner le CPU à un autre processus plus prioritaire. Exemple : SRTF, Round Robin.

L'Algorithme SRTF

1) Définition:

SRTF (Shortest Remaining Time First) est la version préemptive de l'algorithme SJF (Shortest Job First). À chaque unité de temps, le processeur est alloué au processus qui possède le temps d'exécution restant le plus court.

2) Formules de calcul:

Completion Time (CT) = moment où le processus se termine

Turnaround Time (TAT) = CT – Arrival Time

Waiting Time (WT) = TAT – Burst Time

Average Waiting Time = $\Sigma(WT) / n$

Average Turnaround Time = $\Sigma(TAT) / n$

3) Avantages de SRTF:

- Minimise le temps d'attente moyen

- Optimal pour les systèmes interactifs

- Bonne utilisation du CPU

- Favorise les processus courts

4) Inconvénients de SRTF :

- Peut provoquer la famine (starvation) des processus longs

- Nécessite de connaître les temps d'exécution à l'avance

- Overhead important dû aux changements de contexte fréquents

- Difficile à implémenter dans un système réel

Exemple Illustratif

1) Donnes d'entree

Processus	Temps d'arrive	Temp execution
P1	0	6
P2	1	4
P3	2	2
P4	4	1

2) Resultat

Processus	CT	TAT	WT
P1	13	13	7
P2	7	6	2
P3	4	2	0
P4	5	1	0

Averg waiting time = $(7+2+0+0)/4 = 2.25$

Avg turnaround time = $(13+6+2+1)/4 = 5.5$

Architecture du projet

-Technologies utilisees :

Technologie	version	Role
python	3.x	Implementantation de l'algorithme
My SQL	8.0	Stockage des resultas
Docker	latest	Conteneurisation
Debian	latest	Systeme d'exploitation de base
Mysql-connector-python	latest	Connexion python-MySQL

Docker et Conteneurisation

1) Qu'est-ce que Docker ?

Docker est une plateforme de conteneurisation qui permet d'empaqueter une application avec toutes ses dépendances dans un conteneur isolé. Cela garantit que l'application fonctionne de manière identique sur n'importe quelle machine.

2) Le Dockerfile :

Explication ligne par ligne :

FROM debian:latest → Utilise Debian comme base

RUN apt-get install python3 → Installe Python

WORKDIR /app → Définit le répertoire de travail

COPY → Copie les fichiers dans le conteneur

CMD → Lance le programme au démarrage

3) Docker Compose :

Docker Compose orchestre deux conteneurs :

Conteneur 1 — MySQL (db) :

Image officielle MySQL 8.0

Crée automatiquement la base srtf_db

Expose le port 3306

Vérifie sa santé avant de démarrer Python

Conteneur 2 — Python (app) :

Construit depuis notre Dockerfile

Dépend du conteneur MySQL

Mode interactif pour la saisie utilisateur

3) Communication entre conteneurs :

Communication entre conteneurs

Les deux conteneurs communiquent via le réseau Docker interne.

Python utilise le nom db comme hôte MySQL, ce qui est résolu automatiquement par Docker.

Page 8 — Tests et Résultats